



23CSE203

DATA STRUCTURES AND ALGORITHM

Submitted by	
Name	J.Rudrateja
Roll.no	AV.SC.U4CSE25114
Section	CSE-B

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Credit	5
--------	---

Index

S.no	Title	Page.no	Remark
1.	E) A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. E) Bank calculates the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power), where P=principal growth factor, n=no.of years. E) Write a recursive program to print the first n terms of fibonacci series E) An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list. E) In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.	4-13	
2.	B) write a python program to perform linear search on the given input B) Write a python program to perform binary search on the given	14-18	

	input. Verify by taking sorted array and unsorted array as a list.		
3.	A) write a python program to implement bubble sort, where the elements are taken from user. B) write a python program to implement selection sort, where the elements are taken from user. C) write a python program to implement insertion sort, where the elements are taken from user.	19-	

Week-1

A)LAUNCH

Aim: A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def countdown(n):  
    if n == 0:  
        print("launch")  
    else:  
        print(n)  
        countdown(n - 1)  
  
n = int(input("Enter the starting number: "))  
countdown(n)  
|
```

Output:

==== RESTART: C:/Users/rudra/A:

Enter the starting number: 24

24

23

22

21

20

19

18

17

16

15

14

13

12

11

10

9

8

7

6

5

4

3

2

1

launch

Explanation:

- It takes a number as input from the user and stores it in num.
- It creates a function named launch().
- It starts the countdown from 10 by assigning i = 10.
- It prints the value of i as long as num <= i.
- It decreases the value of i by 1 after each iteration.
- When the loop ends, it prints "Rocket launched" and executes the function using launch().
- For negative input, the function keeps decreasing instead of reaching 0, causing infinite recursion and eventually a Recursion Error.

Error table:

s.no	Error obtained	Rectification
1.	Infinite loop	Iteration at correct place
2.	Calling function at wrong place	Placed with correct syntax

B)POWER

Aim: Bank calculates the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where P=principal growth factor, n=no. of years.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def power(p, n):
    if n == 0:
        return 1
    elif n < 0:
        return 1 / power(p, -n)
    else:
        return p * power(p, n - 1)

p = float(input("Enter the principal growth factor: "))
n = int(input("Enter the number of years: "))

print("Power =", power(p, n))
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local
Enter the principal growth factor: 3
Enter the number of years: 2
Power = 9.0
>|
```

Explanation:

- The program takes the principal growth factor p and number of years n as input.
- It defines a recursive function $\text{power}(p, n)$.
- If n is 0, the function returns 1 (base case).
- Otherwise, it returns $p * \text{power}(p, n-1)$.
- The recursive calls continue until n becomes 0.
- The final value of p^n is displayed as the output.
- For a negative exponent, calculate the reciprocal instead of continuing recursion downward indefinitely.
- Error table:

s.no	Error obtained	Rectification
1.	Return 0	Return 1
2.	print	return

C)FIBANOCCHI

Aim: Write a recursive program to print the first n terms of fibanocci series Program:

```
dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)
File Edit Format Run Options Window Help

def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ====
Enter number of terms: 10
0 1 1 2 3 5 8 13 21 34
```

Explanation:

- For entering the number as input from user we defined as n
- We declared variable for formula.
- If input number is less than one then it return "1"
- Or it will goes under fibanocci iteration.
- If x is not present in the list, recursion reaches the end and returns False. Error:

s.no	Error obtained	Rectification
1.	Forgot to declare $n \leq 1$	rectified
2.	Formula mistake	rectified

D)EMPLOYEE ID

Aim: An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list.

Program:

```
dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)
File Edit Format Run Options Window Help
def search(emp_list, x, i=0):
    if i == len(emp_list):
        return False

    if emp_list[i] == x:
        return True

    return search(emp_list, x, i + 1)

emp_list = [101, 205, 310, 415, 520]

x = int(input("Enter employee ID to search: "))

if search(emp_list, x):
    print("Employee ID found.")
else:
    print("Employee ID not found.")
```

Output:

```
=== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py =
Enter employee ID to search: 101
Employee ID found.
```

Explanation:

- The program creates a function to search for an employee ID in a list.
- It checks one employee ID at a time, starting from the first ID.

- If the entered employee ID matches the current ID, it returns True.
- If it does not match, the function moves to the next ID using recursion.
- If all employee IDs are checked and no match is found, it returns False.
- Finally, the program displays "Employee ID found." if the ID exists; otherwise, it displays "Employee ID not found."
- Error:

s.no	Error obtained	Rectification
1.	Emp lst	Emp_list
2.	print	Return

E) Recursive Factorial :

Aim: In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number. Program:

```
*dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)*
File Edit Format Run Options Window Help
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

n = int(input("Enter a number: "))

if n < 0:
    print("Factorial is not defined for negative numbers.")
else:
    print("Factorial =", factorial(n))
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ====
Enter a number: 4
Factorial = 24
|
```

Explanation:

- The program defines a recursive function fact() to calculate the factorial of a number.
- It takes a number as input from the user.
- If the number is 1, the function returns 1 (base case).
- Otherwise, it multiplies the number by the factorial of the previous number (n-1).
- The recursive calls continue until the number becomes 1.
- Finally, the program prints the factorial of the given number.
- Negative numbers do not have a factorial, so the program should handle them separately instead of recursing forever.

Error table:

s.no	Error obtained	Rectification
1.	n+1	n-1
2.	Syntax error	Rectified

WEEK-2

A) LINEARSEARCH

AIM: Write a python program to perform linear search on the given input.

Program:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = list(map(int, input("Enter elements separated by space: ").split()))
key = int(input("Enter the element to search: "))

result = linear_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py =
Enter elements separated by space: 3 3
Enter the element to search: 3
Element found at index: 0
>
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complicity is $O(n)$.
- The target element may be found at beginning or last or at middle.
- If the key is greater than every element in the sorted array, the search reaches the end and returns -1. Error table:

s.no	Error obtained	Rectification
1.	Return placed in wrong place	Corrected later

2.	Calling the function	Rectified
----	----------------------	-----------

B) BINARY SEARCH

Aim: Write a python program to perform binary search on the list.

Program: Unsorted

```
dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)
File Edit Format Run Options Window Help
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

arr = list(map(int, input("Enter elements separated by space: ").split()))
arr.sort()

print("Sorted array:", arr)

key = int(input("Enter the element to search: "))
result = binary_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ====
Enter elements separated by space: 12
Sorted array: [12]
Enter the element to search: 1
Element not found
|
```

Program: Sorted

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

arr = list(map(int, input("Enter sorted elements separated by space: ").split()))
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Py
Enter sorted elements separated by space: 1 2 3 4 5 6
Enter the element to search: 4
Element found at index: 3
```

Explanation:

- The binary search should always be executed in sorted order.
- Either it can be ascending or descending order.
- So the program is in unsorted order so we need to do sorting first later the binary search is executed.
- Error table:

s.no	Error obtained	Rectification
------	----------------	---------------

1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A) Bobble sort:

AIM: write a python program to implement bubble sort, where the elements are taken from user.

Program:

```
*dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)*
File Edit Format Run Options Window Help

def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

n = int(input("Enter no. of elements: "))
arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

bubble_sort(arr)

print("Sorted Array:")

for element in arr:
    print(element, end=" ")
```



```

===== RESTART: C:/Users/rudra/
Enter no. of elements: 4
Enter the elements:
1
2
6
8
Sorted Array:
1 2 6 8

```

Output:

Explanation:

- Bubble sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program may contain elements in **unsorted order**, so adjacent elements are compared and swapped if they are in the wrong order.
- This process is repeated for multiple passes until the **entire array is sorted**.
- For $n = 0$, the program now prints the error message without trying to use an undefined arr.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B)insertion sort:

AIM: write a python program to implement insertion sort, where the elements are taken from user.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key

n = int(input("Enter no. of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

insertion_sort(arr)

print("Sorted Array:")

for element in arr:
    print(element, end=" ")
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py
Enter no. of elements: 2
Enter the elements:
1
2
Sorted Array:
1 2
```

Explanation:

- Insertion sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program takes one element at a time and **inserts it into its correct position** in the already sorted part.
- This process is repeated until **all elements are sorted**.
- When the number of elements is zero, the array is empty and insertion sort simply performs no iterations.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

C) Selection sort:

AIM: write a python program to implement selection sort, where the elements are taken from user.

Program:

dsa.py - C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py (3.13.7)

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

n = int(input("Enter no. of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

selection_sort(arr)

print("Sorted Array:")

for element in arr:
    print(element, end=" ")
```

Output:

```
==== RESTART: C:/Users/rudra/AppData/Local/Programs/Python/Python313/dsa.py ===
Enter no. of elements: 3
Enter the elements:
5
4
3
Sorted Array:
3 4 5
```

Explanation:

- Selection sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program finds the **smallest element** from the unsorted part and swaps it with the first element of that part.
- This process is repeated until the **entire array is sorted**.
- When $n = 0$, the array is empty and selection sort performs no iterations, so the program runs normally.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified